Session 08: Validation

Rikkei Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

1. Tổng quan về Validation trong ứng dụng Web
2. Tổng quan Bean Validation & tích hợp với Spring
3. Hiển thị lỗi với BindingResult trong Form
4. Giới thiệu về Custom Annotation Validator

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

1

Tổng quan về Validation trong ứng dụng Web

Validation là gì?

Là quá trình kiểm tra dữ liệu đầu vào trước khi xử lý. Mục tiêu đảm bảo dữ liệu đúng định dạng, đầy đủ thông tin và hợp lệ theo yêu cầu nghiệp vụ.

Đúng định dạng

Email phải có @, số điện thoại phải là chữ số, ngày tháng đúng cấu trúc

Đủ thông tin

Các trường bắt buộc không được để trống, dữ liệu phải có giá trị

Hợp lệ nghiệp vụ

Tuổi phải lớn hơn 0, số lượng không âm, mã sản phẩm quy ước

Validation áp dụng cho mọi điểm nhận dữ liệu: form nhập liệu trên giao diện và API request từ client hoặc hệ thống bên ngoài

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Client-side vs Server-side Validation

Hai tầng kiểm tra dữ liệu hoạt động bổ trợ nhau trong một ứng dụng web hiện đại. Mỗi tầng có vai trò và giới hạn riêng.

Client-side (Frontend)

Validation chạy ngay trên trình duyệt của người dùng, trước khi dữ liệu được gửi lên server.

- Phản hồi tức thì, không chờ server
- Cải thiện trải nghiệm người dùng
- Giảm tải cho backend với lỗi đơn giản
- Ví dụ: HTML5 required, JS validation

Server-side (Backend)

Validation chạy trên server sau khi nhận dữ liệu. Đây là tuyến phòng thủ bắt buộc và không thể bỏ qua.

- Không thể bị vô hiệu hóa bởi người dùng
- Kiểm tra được login nghiệp vụ phức tạp
- Truy cập database để xác minh dữ liệu
- Là lớp bảo vệ duy nhất đáng tin cậy

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Vì sao không thể tin Client?

Client-side validation chỉ là lớp hỗ trợ UX - không phải bảo mật. Người dùng hoàn toàn có thể vượt qua bằng nhiều cách khác nhau.

Tắt JavaScript

Toàn bộ validation viết bằng JavaScript bị vô hiệu hóa. Form vẫn submit được với dữ liệu bất kỳ.

Gửi request bằng Postman

Bỏ qua hoàn toàn giao diện web. Dữ liệu được gửi thẳng đến API endpoint mà không qua bất kỳ kiểm tra nào ở FE

Chỉnh sửa HTML trực tiếp

DevTools cho phép xóa thuộc tính required, thay đổi maxlength hoặc type của input field trong vài giây.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Rủi ro khi không Validate phía Server

Bỏ qua server-side validation là một trong những lỗi bảo mật nghiêm trọng nhất. Hậu quả có thể ảnh hưởng toàn bộ hệ thống.

SQL Injection

Kẻ tấn công chèn câu lệnh SQL vào dữ liệu đầu vào, đọc hoặc xóa toàn bộ database

XSS Attack

Script độc hại được lưu vào database và thực thi trên trình duyệt của người dùng khác

Dữ liệu sai lệnh

Dữ liệu không hợp lệ gây ra kết quả tính toán sai, báo cáo không chính xác

Crash hệ thống

NullPointerException, ... khiến ứng dụng sập, ảnh hưởng tất cả người dùng

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

2

Tổng quan Bean Validation & Tích hợp với Spring

Bean Validation - JSR 380

Bean Validation (JSR 380) là chuẩn Java EE định nghĩa cách validate dữ liệu thông qua annotation. Lập trình viên khai báo rule trực tiếp trên các field của object.

1

SQL Injection

Đặt annotation như **@NotNull, @Email** trực tiếp lên field trong DTO

2

Validation tự động

Framework đọc annotation và thực hiện kiểm tra mà không cần code thủ công.

3

Thông báo lỗi rõ ràng

Mỗi annotation có message mặc định có thể tùy chỉnh theo yêu cầu.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Hibernate Validator - Implementation thực tế

Hibernate Validator là implementation phổ biến nhất của Bean Validator. Spring Boot đã tích hợp sẵn Hibernate Validator, thêm dependency là có thể sử dụng.

Thêm dependency vào build.gradle

```
dependencies {
    implementation 'jakarta.validation:jakarta.validation-api:4.0.0-M1'
    implementation ‘org.hibernate.validator:hibernate-validator:9.1.0.Final’
}
```

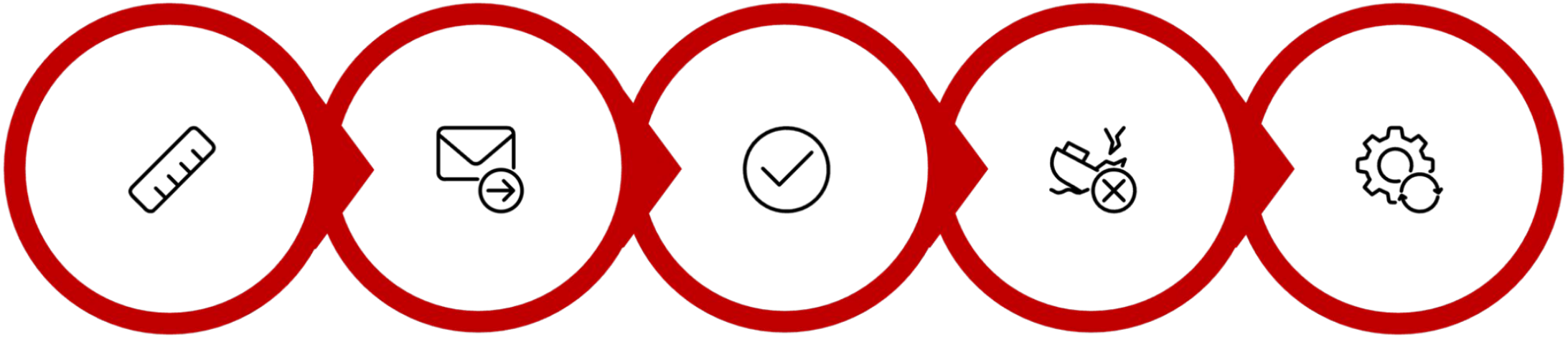
Lợi ích khi dùng Hibernate Validator

- Hỗ trợ đầy đủ chuẩn JSR 380
- Tích hợp liền mạch với Spring MVC
- Hỗ trợ i18n cho thống báo lỗi
- Có thể mở rộng với custom validator

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Cách hoạt động của Bean Validation

Spring tự động kích hoạt quá trình validation khi nhận request. Luồng xử lý như sau:



Định nghĩa rule | **Client gửi** | **Spring validate** | **Trả lỗi** | **Xử lý tiếp**

Toàn bộ quá trình diễn ra tự động bên trong **Spring MVC**. Chỉ cần khai báo rule một lần trong **DTO** và thêm annotation **@Valid** trong **Controller** là đủ.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

DTO - Nơi đặt Validation Rules

DTO (**Data Transfer Object**) là đối tượng dùng để nhận dữ liệu từ client. Đây là nơi tập trung tất cả validation rules, giup tách biệt logic nghiệp vụ khỏi logic kiểm tra dữ liệu.

Ví dụ RegisterDTO

```
public class RegisterDTO {
    @NotBlank(message = "Username bắt buộc")
    @Size(min = 3, max = 50)
    private String username;

    @NotBlank(message = "Email bắt buộc")
    @Email(message = "Email không hợp lệ")
    private String email;

    @NotBlank
    @Size(min = 8, message = "Tối thiểu 8 ký tự")
    private String password;
}
```

Tại sao dùng DTO?

- Tách biệt dữ liệu đầu vào với Entity trong Database
- Bảo mật: không expose model
- Dễ kiểm soát fields được phép nhận
- Rule validation tập trung một chỗ

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Các Annotation Validation phổ biến

Bean Validation cung cấp bộ **annotation** phong phú để xử lý hầu hết các trường hợp kiểm tra dữ liệu thông thường.

1

@NotNull

- Giá trị không được null.
- Chấp nhận chuỗi rỗng “”.
- Dùng cho mọi dữ liệu.

```
@NotNull
private String name;
```

2

@NotEmpty

- Không được null, không được rỗng
- Áp dụng cho String, Collection, Array
- Chuỗi chỉ có khoảng trắng vẫn pass

```
@NotEmpty
private String title;
```

3

@NotBlank

- Không được null, không rỗng và không chỉ chứa khoảng trắng.
- Nghiêm ngặt nhất nhóm Not
- Chỉ dùng cho String

```
@NotBlank
private String username;
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Các Annotation Validation phổ biến

Bean Validation cung cấp các annotation để validate định dạng và giới hạn giá trị số

@Email

- Kiểm tra đúng định dạng địa chỉ email
- Phải có ký tự **@** và **tên miền** hợp lệ

```
@Email
private String email;
```

@Size

- Giới hạn độ dài hoặc kích thước collection.
- Có thể chỉ định min, max hoặc cả hai

```
@Size(min=3, max=50)
private String name;
```

@Min / @Max

- Giới hạn giá trị số nguyên **@Min(1)** đảm bảo lớn hơn hoặc bằng 1
- Dùng cho **int, long, BigDecimal**

```
@Min(1) @Max(100)
private int age;
```

@Pattern

- Kiểm tra chuỗi khớp với biểu thức chính quy (regex)
- Linh hoạt cho mọi định dạng tùy chỉnh

```
@Pattern(regexp="^[0-9]{10}$")
private String phone;
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

@Valid và @Validated - Kích hoạt Validation

Annotation trong **DTO** chỉ là khai bao rule. Để Spring thực sự chạy validation, cần thêm **@Valid** vào tham số trong **Controller**

Cách sử dụng trong Controller

```
@PostMapping("/register")
public String register(
    @Valid @ModelAttribute RegisterDTO dto,
    BindingResult result) {

  if (result.hasErrors()) {
    return "register-form";
  }
  // Xử lý khi dữ liệu hợp lệ
  userService.save(dto);
  return "redirect:/success";
}
```

Lưu ý quan trọng

Nếu thiếu **@Valid** trước object DTO, toàn bộ các annotation validation trong DTO sẽ không được kích hoạt - không có lỗi nào được phát hiện dù dữ liệu sai.

@Validated là phiên bản của Spring, hỗ trợ thêm validation theo nhóm (groups)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Lỗi thường gặp khi dùng Validation

Nhiều lập trình viên mới gặp những lỗi phổ biến khiến validation không hoạt động đúng. Nhận biết sớm sẽ tiết kiệm rất nhiều thời gian debug.

Quên @Valid trong Controller

Validation không chạy. Dữ liệu sai vẫn được xử lý. Lỗi khó phát hiện vì không có exception

Đặt sai annotation

Dùng **@NotNull** thay vì **@NotBlank** cho String hoặc chỉ có khoảng trắng vẫn pass qua

DTO không đúng cấu trúc

Thiếu **getter/setter** khiến Spring không đọc được giá trị. Validation chạy nhưng không lỗi

Không xử lý lỗi trả về

Thiếu **BindingResult** khiến Spring ném exception **400 Bad Request** mà không báo lỗi

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

3

Hiển thị lỗi với Binding Result trong Form

Xử lý lỗi trong Spring MVC

Mặc định khi validation thất bại và không có BindingResult, Spring sẽ ném exception và trả về lỗi HTTP 400 Bad Request - thô và không thân thiện với người dùng cuối.

Không có BindingResult

Spring ném MethodArgumentNotValidException.

Người dùng thấy trang lỗi hoặc JSON lỗi kỹ thuật

Có BindingResult

Lỗi được bắt lại và lưu trong object.

Developer chủ động kiểm tra và quyết định cách xử lý.

Trả lại form với thông báo lỗi rõ ràng

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

BindingResult - Lưu trữ kết quả Validation

BindingResult là interface của Spring chứa toàn bộ kết quả sau khi validation chạy xong. Nó cho phép kiểm tra lỗi và xử lý linh hoạt thay vì để hệ thống crash

Sử dụng BindingResult đúng cách

```
@PostMapping("/register")
public String register(
    @Valid @ModelAttribute RegisterDTO dto,
    BindingResult bindingResult,
    Model model) {

  // Kiểm tra có lỗi không
  if (bindingResult.hasErrors()) {
    // Trả lại form, giữ nguyên dữ liệu
    return "register-form";
  }

  userService.save(dto);
  return "redirect:/success";
}
```

Nguyên tắc bắt buộc

BindingResult phải đặt ngay sau @Valid object. Nếu có tham số khác xen vào giữa Spring sẽ không liên kết dùng và vấn ném exception 400.

register(@Valid RegisterDTO dto, BindingResult result);

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Hiển thị lỗi trên giao diện với Thymeleaf

Thymeleaf cung cấp các expression đặc biệt để hiển thị thông báo lỗi validation trực tiếp trên form, giúp người dùng biết chính xác trường nào sai và sai như thế nào.

Template Thymeleaf với hiển thị lỗi

```
<form th:action="@{/register}" th:object="${registerDTO}" method="post">
  <div>
          <input type="text" th:field="*{username}" />
         <span th:if="*{#fields.hasErrors('username')}"
                th:errors="*{username}"
                class="error">
         </span>
  </div>
  <div>
         <input type="email" th:field="*{email}" />
         <span th:errors="*{email}"></span>
  </div>
</form>
```

Các expression quan trọng

- **th:errors="*{field}"** - hiển thị thông báo lỗi của field
- **#fields.hasErrors('field')** - kiểm tra field có lỗi không
- **th:if** - chỉ hiển thị khi có lỗi

Lợi ích UX

- Người dùng thấy lỗi ngay tại chỗ
- Dữ liệu đã nhập được giữ lại
- Không cần nhập lại dữ liệu

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

4 Custom Annotation Validator

Khi nào cần Custom Validation?

Bean Validation chuẩn xử lý tốt các rule phổ biến, nhưng trong thực tế có nhiều yêu cầu nghiệp vụ đặc thù mà annotation mặc định không đáp ứng được.

Rule đặc thù của hệ thống

Mã sinh viên phải bắt đầu bằng BK_, mã sản phẩm theo định dạng riêng của công ty

Kiểm tra tính duy nhất

Username, email không được trùng với dữ liệu đã có trong DB - cần truy vấn repository

Logic phức tạp nhiều trường

Ngày kết thúc phải sau ngày bắt đầu, tổng số lượng không vượt quá tồn kho, giá sale thấp hơn giá gốc

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Cấu trúc một Custom Validator

Mỗi custom validation gồm hai thành phần cốt lõi hoạt động cùng nhau: một annotation để khai báo và một validator class để xử lý logic

Bước 1: Tạo Annotation (@interface)

Định nghĩa annotation tùy chỉnh với `@interface`. Khai báo các thuộc tính như `message`, `groups`, `payload`. Liên kết với validator class qua `@Constraint(validatedBy = ...)`.

Bước 2: Tạo Validator Class

Implement interface `ConstraintValidator<A, T>` với hai type parameter: annotation type và kiểu dữ liệu cần validate. Override phương thức `isValid()` chứa toàn bộ logic kiểm tra.

Hai thành phần được kết nối qua `@Constraint(validatedBy = MyValidator.class)` trong annotation definition.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Tạo một Custom Annotation

Ví dụ **@ValidStudentCode** để kiểm tra mã sinh viên phải bắt đầu bằng **BK_**

Định nghĩa annotation

```
@Documented
@Constraint(validatedBy =
StudentCodeValidator.class)
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidStudentCode {

  String message()
        default "Mã sinh viên phải bắt đầu BK_";
  Class<?>[] groups() default {};
  Class<? extends Payload>[] payload()
        default {};
}
```

Giải thích các annotation meta

- `@Constraint` — liên kết với class xử lý logic
- `@Target` — nơi có thể dùng annotation (FIELD, METHOD, TYPE...)
- `@Retention` — giữ annotation đến runtime
- `message()` — thông báo lỗi mặc định
- `groups()`, `payload()` — bắt buộc theo chuẩn JSR 380

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Implement ConstraintValidator

Class validator implement interface ConstraintValidator, logic kiểm tra trong isValid()

class Validator đơn giản

```
public class StudentCodeValidator implements
        ConstraintValidator<ValidStudentCode, String> {
  @Override
  public void initialize(ValidStudentCode annotation) {
        // Khởi tạo nếu cần đọc thuộc tính
  }
  @Override
  public boolean isValid(String value,
ConstraintValidatorContext context) {
        if (value == null) return false;
        // Logic kiểm tra nghiệp vụ
        return value.startsWith("BK_");
  }
}
```

Chi tiết phương thức isValid()

Tham số value là giá trị cần kiểm tra

Tham số context dùng để tùy chỉnh thông báo lỗi động

Trả về true - dữ liệu hợp lệ

Trả về false - dữ liệu không hợp lệ, Spring tạo lỗi

Annotation @ValidStudentCode có thể dùng ngay trên bất kỳ field String nào trong DTO

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Inject Repository vào Validator

Validator class có thể inject các dependency như Repository để kiểm tra dữ liệu với DB

Validator kiểm tra username unique

```
@Component
public class UniqueUsernameValidator implements
        ConstraintValidator<UniqueUsername, String> {
  // Inject Spring Repository
  @Autowired
  private UserRepository userRepository;
  @Override
  public boolean isValid(String username, ConstraintValidatorContext context) {
        if (username == null) return true;
        // Gọi database kiểm tra
        return !userRepository
        .existsByUsername(username);
  }
}
```

Sử dụng trong DTO

```
public class RegisterDTO {
  @NotBlank
  @Size(min = 3, max = 30)
  @UniqueUsername
  private String username;
  @NotBlank
  @Email
  private String email;
  @NotBlank
  @Size(min = 8)
  private String password;
}
```

Spring tự động inject **UserRepository** vào validator khi container khởi động. Không cần cấu hình thêm.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- Validation là yêu cầu bắt buộc của mọi ứng dụng web.
- Spring Boot cung cấp đầy đủ công cụ để triển khai validation sạch, hiệu quả và dễ bảo trì
- Bean Validation dùng annotation thay if-else, giúp giảm boilerplate, chuẩn hóa code, dễ đọc và bảo trì hơn đáng kể
- BindingResult giúp bắt lỗi, cho phép trả lại form với thông báo lỗi thân thiện, cải thiện trải nghiệm người dùng
- Custom Validator xử lý nghiệp vụ không có trong chuẩn
- Có thể inject Spring Bean, truy cập database linh hoạt

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN